



🦂 The Workflow Factory

Thin harness, fat skills, and the five primitives that scale a workflow platform without hardcoding any workflow.

“ Mythology had a god of messengers because the message must arrive whole. We built one. ”

One harness. Many recipes. Every recipe a YAML file. Every capability a Package. Every contract a schema. Every skill a versioned procedure. Add a domain in a day; add a workflow in an afternoon. The harness never knows what kind of work it's doing — and that is exactly why it can do anything.

IN ONE PAGE

The thesis. A workflow platform that hardcodes its workflows is a fixed-purpose tool. A workflow platform that lets workflows be authored as data — declarative recipes, versioned skills, installable packages — is an extensible factory. The factory's value compounds with every workflow added, because the harness never grows.

The five primitives. Package (the unit of installation), Recipe (the orchestrator), Skill (the unit of capability), Ingredient (per-stage behavior modifier), Extension (workflow-spanning capability). One scan loop, one registry, one validation protocol. Adding a new workflow domain — marketing, research, support, legal — requires zero changes to the harness. Author a Package, declare its contributions, restart the server. The factory routes the new contributions automatically.

The integration story. Every external system — LLMs, observability stacks, dashboards, version control, message buses — connects through a port + adapter pair. Swap an adapter, change nothing else. The harness has no dependency on Postgres, no dependency on Hermes-the-CLI, no dependency on any specific LLM. It has dependencies on *shapes*: `IStoryRepository`, `IAgentHarness`, `IEventEmitter`, `IGitAdapter`. The adapters are interchangeable.

The scale story. The harness is bounded by doctrine: it does five things and nothing more. As the catalog of workflows grows, the harness stays the same size. A team of one can ship a new domain in a week. A team of fifty can ship fifty domains in parallel. *The factory floor never expands; only the products do.*

CONTENTS

1. The Promise — what a workflow factory is, and why it matters
2. Doctrine — Thin Harness, Fat Skills, and the boundary that holds
3. The Five Primitives — Package, Recipe, Skill, Ingredient, Extension
4. How They Compose — the Resolved Recipe and runtime guarantees
5. Integrations — ports, adapters, and the contract surface
6. The Plugin Architecture — adding a capability without inflating the core
7. A Day in the Life — three workflows, one harness, end-to-end
8. Scaling — the math of "the harness stays the same size"
9. Operating Procedure — how to extend the factory
10. Closing — the workflow factory's promise

🍷 I. The Promise

A workflow factory is a system that runs workflows it does not understand. A team can ship a new workflow on it without modifying the system itself.

The promise is concrete: a marketing operations team should be able to add a campaign-approval workflow tomorrow, with its own roles (strategist, copywriter, brand-reviewer, legal, distribution) and its own quality bars, by writing YAML and Markdown files. The system that runs this workflow already runs a software-development workflow with completely different roles (architect, implementer, reviewer, tester, releaser) and completely different quality bars. The harness — the runtime — is identical for both. It does not branch on "is this dev or marketing." It does not know.

That sentence — "*it does not know*" — is the entire architectural commitment. Everything in this paper is the structure that makes that sentence true.

✦ THE LITMUS TEST

Adding a new workflow domain requires zero changes to `src/pipeline/`, `src/services/dispatcher.ts`, `src/adapters/agent/`, or any harness internal. If your change requires editing those, you've broken the doctrine. Either the change belongs in a Package (extension), or the harness is missing a primitive that should have been declared, not coded.

Why this matters

Most workflow systems ship as fixed-purpose tools — a build pipeline, a content management workflow, a customer-support escalation tree — and grow brittle as their authors keep adding "just one more case" to the core code. The core inflates. Every change risks a regression in an unrelated workflow. The team that built the platform becomes a bottleneck because they own the only code that knows what each workflow does.

The workflow factory inverts this. The team that builds the platform owns the harness. Workflow authors own their Packages. The two are decoupled: a Package author cannot break the harness, and a harness author cannot break a workflow except by changing a port contract — which is a versioned event, surfaced loudly. *The platform team scales sub-linearly with the workflow catalog.*

🍷 II. The Doctrine

Skills carry process. Adapters perform deterministic execution. The harness orchestrates and routes.

Three layers, in strict order, with a bright line between each:

I

Latent layer — fat skills

Markdown files in the repo. Each skill is a versioned procedure with frontmatter (name, version, parameters, output schema, references) and a body that walks the procedure with examples, anti-patterns, failure modes. A skill answers five questions for the agent that runs it: *Who am I? What's my contract? How do I do it? When am I done? What if I fail?* Skills are domain-specific. A research skill knows research. A marketing skill knows marketing. The harness knows neither.

II

Deterministic layer — adapters

TypeScript classes implementing typed ports. Each adapter does one job: write to Postgres, run a Hermes subprocess, push a git ref, emit a Prometheus metric, send to a Slack channel. Adapters are *boring*. They have no judgment. They take input, produce output, fail loud on errors. They are interchangeable per port — swap `PostgresStoryRepository` for `SqliteStoryRepository`, the harness doesn't notice. Adapters do **not** embed domain knowledge.

III

Orchestration layer — thin harness

The smallest possible runtime that connects skills to adapters. It does **five things**: (1) run the dispatch loop; (2) read and write files; (3) manage context; (4) enforce safety (auth, rate limits, idempotency, named gates); (5) discover and route Package contributions. It does not have a hardcoded list of roles. It does not encode "if marketing then..." branches. It does not know what kind of work the skill is doing. It dispatches and validates contracts. *That is all.*

If you cannot describe a change as "I added a Package" or "I swapped an adapter" or "I extended a port contract," then you are about to inflate the harness. Pause and find the primitive you missed.



III. The Five Primitives

One installable unit (Package) declares contributions of four types (Recipe, Skill, Ingredient, Extension). Five primitives total. Every capability the platform ever ships fits one of these shapes.

Package — the installable unit

I

A directory containing a `package.yaml` manifest that declares what the Package contributes. Packages depend on other Packages (version-pinned). The harness has one PackageRegistry that scans `packages/` at boot and loads every manifest. Examples shipped today: `ops-core`, `ops-dev-suite`. Examples planned: `ops-marketing-suite`, `ops-research-suite`, `ops-support-suite`.

```
# packages/ops-marketing-suite/package.yaml
name: ops-marketing-suite
version: 1.2.0
requires:
  - name: ops-core version: ^1.0.0
primitives:
  recipes:    [campaign-launch, approval-flow]
  skills:     [strategist-brief, copywriter-draft, brand-review]
  ingredients: [postiz-publish]
  extensions: [slack-notifications]
```

Inspired by npm packages, Rails Engines, Django apps, VSCode extensions. One scan loop, one registry, one install protocol.

Recipe — the orchestrator

II

YAML declaring which Packages are active and what stages run, in order. A Recipe references skills by name; it does not contain procedure logic. Recipes are immutable on disk — no runtime mutation, fully auditable. Different recipes have different role compositions. The dev recipe runs five roles. A marketing campaign recipe runs four. A research recipe might run two stages, then loop. Every recipe is a different orchestration of the same skill catalog.

```
name: campaign-launch
packages: [ops-marketing-suite, slack-notifications]
stages:
  - role: strategist
    skill: strategist-brief
    cardinality: per-epic
    exit: complete
  - role: copywriter
    skill: copywriter-draft
    cardinality: per-story
    exit: verify
  - role: brand-reviewer
    skill: brand-review
    cardinality: per-story
    exit: verify
    routeBack:
      tone-mismatch: copywriter
  - role: distributor
    skill: postiz-distribute
    ingredients: [postiz-publish]
    cardinality: per-story
    exit: complete
```

Skill — the unit of capability

III

A Markdown file (`SKILL.md`) with frontmatter and a body. One skill, one job. Skills are stateless leaf nodes — they do not invoke other skills. They take a typed input, follow a procedure, return a typed output. The 12-section Google-tier template (Purpose, Contract, Inputs, Procedure, Quality Bar, Output Format, Failure Modes, Self-Review, References, Examples, Anti-Patterns, When Scope Expands) ensures every skill is fat enough to operate without falling back to the harness for guidance.

Skills are domain-agnostic in implementation: every skill receives a `BootContext` envelope at dispatch time and produces output validated by `ContractGate` against a JSON Schema. The skill body is what makes it domain-specific. `brand-review` knows brand voice; `strategist-brief` knows campaign positioning; `code-reviewer` knows software craftsmanship. The harness validates the schema, not the substance.

Ingredient — per-stage behavior modifier

IV

A TypeScript manifest declaring stage-scoped modifications: gate hooks, lifecycle hooks, schema contributions, side-effect declarations. Ingredients are how a recipe says "this stage produces files that need to be committed" or "this stage publishes to Postiz" without the harness needing to know what publishing means. Resolution is precedence-ordered: explicit YAML > skill-schema-implicit > parent-recipe-inherited.

Reserved names like `code-deliverables`, `git-commit`, `git-merge`, `postiz-publish`, `env-required`, `tool-reachable`, `auth-scope` are first-class declarations the harness routes to the right adapter without pattern-matching.

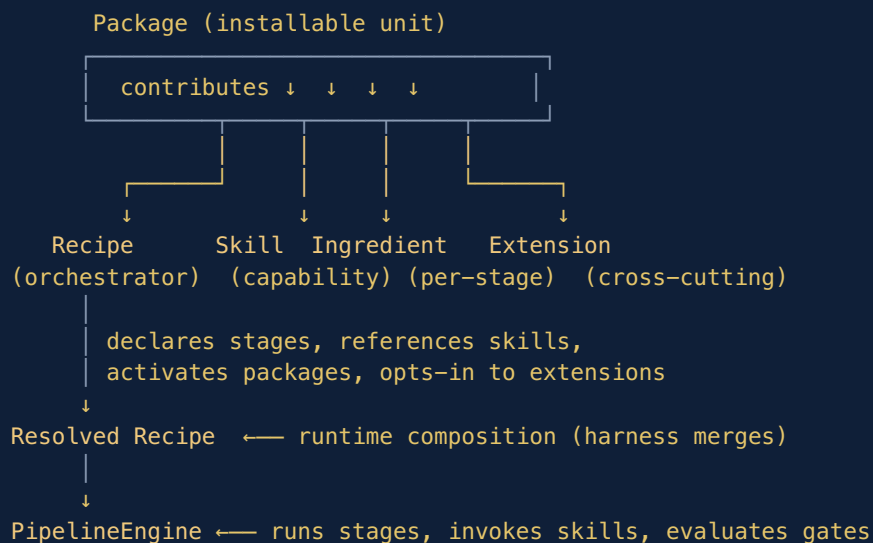
Extension — workflow-spanning capability



A TypeScript manifest declaring cross-cutting behavior: config schema, context namespace, workflow-boundary hooks (`onWorkflowStart` , `onWorkflowComplete` , `onWorkflowFailure`), bundled ingredients, gates, UI fragments, telemetry. Extensions are domain-agnostic — the same Extension (e.g. `slack-notifications`) works in dev, marketing, research, support workflows. Examples designed: `github-integration` , `quality-gates` , `observability` , `slack-notifications` , `postiz-scheduler` , `notion-connector` .

An Extension is the platform's pluggable cross-cutting layer. Want every recipe to notify Slack on failure? Install `slack-notifications` . Want every recipe to emit Prometheus metrics? Install `observability` . The harness routes Extension contributions to the right lifecycle hooks generically.

How they relate



The Recipe is the only primitive that *references* other primitives. Skills, Ingredients, and Extensions are passive: they declare what they provide; they do not know which Recipes use them. This is the inversion that makes the system extensible. A skill author does not need to know the recipe catalog. A recipe author picks from the skill catalog. The harness merges.

🔗 IV. How They Compose — The Resolved Recipe

The Recipe YAML is static. The Resolved Recipe is the runtime artifact — the Recipe merged with all active Packages' contributions, producing a single executable plan.

At dispatch time, the harness does not consult the Recipe YAML directly. It consults the *Resolved Recipe*, which is the result of:

1. **Activate Packages.** Read the Recipe's `packages: [...]` list. For each, locate its manifest in the PackageRegistry. Verify dependencies (transitively, version-pinned). Reject on conflict or missing dep.
2. **Resolve stages.** For each stage, look up the named skill in the SkillRegistry. Inline its `outputSchema` and frontmatter. Resolve ingredients (precedence: explicit > implicit > inherited). Resolve gate names against the GateRegistry.
3. **Merge extension contributions.** For each active Extension, register its workflow-boundary hooks, context namespace, and bundled ingredients into the Resolved Recipe's hooks list.
4. **Validate.** Every reference resolves. Every schema parses. Every gate name maps to a registered gate. Every ingredient name maps to a known kind. If anything misses, boot fails — fail loud at registration, never silent at dispatch.

The output is reproducible (same inputs → same Resolved Recipe), auditable (can be emitted as JSON before execution), cacheable (hashable), diffable (can show what each Package added).

⚙️ RESOLVED RECIPE SHAPE

```
interface ResolvedRecipe {
  name: string;
  version: string;
  sourceRecipe: string;           // path to base YAML
  packages: string[];             // activated packages
  stages: ResolvedStage[];        // merged + ingredient-resolved
  gates: ResolvedGate[];          // merged from extensions
  context: ContextNamespace[];    // merged extension namespaces
  hooks: WorkflowHook[];          // onStart / onComplete / onFailure
}
```

The pipeline engine executes the Resolved Recipe — never the raw YAML. This makes the "who decides what runs" question crisp: the Recipe author decides (Recipe YAML is the source of truth). The Package author can only contribute capabilities the Recipe opts into via `packages: [...]`. There is no implicit injection. There is no global override.

🍷 V. Integrations — Ports, Adapters, and the Contract Surface

Every external system the platform touches connects through a port. The harness depends on shapes; the shapes have plural implementations.

Hexagonal architecture, pure and applied. The harness's dependencies are interfaces in `src/ports/`. Each interface has at least one production adapter and at least one test fake. Swapping requires changing one DI registration; the rest of the codebase is unchanged.

The integration ports today

PORT	PRODUCTION ADAPTER	WHAT IT ABSTRACTS
<code>IStoryRepository</code>	<code>PostgresStoryRepository</code>	Story CRUD + atomic claim-token verification + aggregate save with outbox
<code>IPipelineStateRepository</code>	<code>PostgresPipelineStateRepository</code>	Epic + feature CRUD, branch tracking, merge metadata
<code>IAgentHarness</code>	<code>HermesAdapter</code> / <code>MockAgentHarness</code>	Spawning agent subprocesses with bootcontext and lifecycle
<code>IGitAdapter</code> / <code>IGitTreeReader</code>	<code>ExecGitAdapter</code>	Branch creation, commits, merges, work-tree reads — bounded surface, no <code>--force</code>
<code>IEventEmitter</code>	<code>OutboxEventEmitter</code>	Typed events written transactionally to <code>pipeline_outbox</code>
<code>IEventBus</code>	<code>InProcEventBus</code> / <code>OutboxWorker</code>	Event subscription + dispatch — drains outbox, fans out to consumers
<code>IBootContextAssembler</code>	<code>BootContextAssembler</code>	Pre-dispatch envelope assembly — five sections, hard-fail validation
<code>IPoolRepository</code>	<code>PostgresPoolRepository</code>	Agent pool membership, role mapping, claim selection
<code>IRateLimiter</code>	<code>RedisRateLimiter</code>	Per-agent and per-role rate limiting
<code>IStallDetector</code>	<code>SessionTailStallDetector</code>	Liveness anomaly detection on Hermes session SQLite
	(boot scanners)	Discovery + validation of declarative artifacts

PORT	PRODUCTION ADAPTER	WHAT IT ABSTRACTS
IRecipeRegistry + ISkillRegistry + IIngredientRegistry		
IFeatureRepository	PostgresFeatureRepository	Feature CRUD, hierarchy

The substitution principle

Each port is a real shape. Each adapter satisfies that shape. Production code resolves the port through DI; the resolved adapter is a singleton (per `instanceCachingFactory` wrapping where stateful). Test code resolves the same port to a fake. The harness has zero conditional branching on adapter identity.

DI SEAM EXAMPLE

```
// Production binding
container.register<IAgentHarness>(TOKENS.AgentHarness, {
  useFactory: instanceCachingFactory<HermesAdapter>((c) => {
    return new HermesAdapter(
      c.resolve(TOKENS.Logger),
      c.resolve(TOKENS.PoolManager),
      undefined,
      c.resolve(TOKENS.StoryRepository),
    );
  }),
});

// Swap to a different LLM harness – change ONE line:
container.register<IAgentHarness>(TOKENS.AgentHarness, {
  useFactory: instanceCachingFactory<ClaudeAgentHarness>((c) => new ClaudeAgentHarness(...))
});
```

This is the structural reason a future migration from Hermes to a different agent harness is a one-day refactor: implement `IAgentHarness` in a new class, change the DI binding, run the test suite. The dispatcher does not change. The recipe registry does not change. The skills do not change. The Postgres adapter does not change.

External integrations

Beyond the core ports, the platform integrates with five categories of external system through Extensions and Ingredients:

✦ LLMS & AGENT RUNTIMES

Hermes (Kimi-k2.6 today; Claude / GPT planned via swap), each behind `IAgentHarness`. Subprocess lifecycle, stdin envelopes, hook-bridged activity events.

✦ VERSION CONTROL

`ExecGitAdapter` behind `IGitAdapter`. Branch-per-epic topology, signed commits, ff-only merges to develop. The adapter has no `--force`, no `--no-verify`, no `--amend`. *The surface area IS the allowed behavior.*

✦ OBSERVABILITY

Outbox + LISTEN/NOTIFY into the EventBus, fanned to SSE for the dashboard, Prometheus for metrics, sqlite-hermes-observability for per-session traces. All extensible — drop in `OpenTelemetryEmitter` as another `IEventEmitter` consumer.

✦ STORAGE

Postgres for durable state (stories, epics, outbox, log entries, pool membership). Redis for rate limits, ephemeral session state, SSE pub/sub. Filesystem for git worktrees + boot file-drops. Each behind a port.

✦ COMMUNICATIONS

Slack via `slack-notifications` Extension. Postiz for scheduled posts via `postiz-scheduler`. Notion via `notion-connector`. Each is an Extension that subscribes to workflow-boundary hooks; none touches harness internals.

✦ IDENTITY & SECRETS

JWT via `JwtAuthService`. Postgres-backed RBAC. AES-encrypted secrets via `AesSecretService`. Bearer tokens for agent-write writes; `claim_token` for state-mutating endpoints (atomic in SQL WHERE).

🍷 VI. The Plugin Architecture

A plugin is a Package that contributes capabilities. The harness loads it; the harness does not understand it.

The PackageRegistry runs at boot. It scans `packages/`. For each manifest:

1. Parse and validate the manifest schema.
2. Resolve dependencies against already-loaded Packages.

3. Register every contribution: skills go to SkillRegistry, recipes to RecipeRegistry, ingredients to IngredientRegistry, extensions to ExtensionRegistry.
4. Initialize Extensions: run their `onLoad` hook so they can register gate kinds, context namespaces, telemetry sinks.
5. Cross-link: any Recipe that `packages: [...]`'s a Package now sees that Package's contributions in its Resolved Recipe.

The bounded contract

A Package can contribute exactly four things — Recipes, Skills, Ingredients, Extensions. It cannot contribute "a thing that touches the dispatcher directly" or "a thing that adds a special case to the orchestrator." Those would be harness inflations, and the doctrine forbids them.

★ THE HARD BOUNDARY

Anything a Package wants to do must be expressible as one of the four contribution kinds. If you find yourself wishing for "just one harness change," what you actually need is a new *port* the harness exposes — and that port is a contract change, an ADR-worthy event, a versioned interface. It is not a Package contribution. *Ports are platform commitments. Packages are plugin contributions.*

Adding a new domain — worked example

A team wants to add a customer-support escalation workflow. The roles are: triage-agent, technical-resolver, customer-comms, qa-supervisor. The workflow should escalate to qa-supervisor on negative customer sentiment.

1. Create `packages/ops-support-suite/package.yaml` declaring four skills, one recipe, one ingredient (`sentiment-score`).
2. Author `skills/triage-agent/SKILL.md` + schema. Same for `technical-resolver`, `customer-comms`, `qa-supervisor`.
3. Author `packages/ops-support-suite/recipes/escalation-flow.yaml` with four stages and a `routeBack` rule on `negative-sentiment`.
4. Author `packages/ops-support-suite/ingredients/sentiment-score.ts` — a tiny manifest declaring it consumes `customer-comms`'s output and adds a sentiment gate hook.
5. Restart the server. PackageRegistry scans, validates, registers. Customer-support recipe is dispatchable.

Lines of harness code changed: zero. Lines of TypeScript added: ~60 (the ingredient manifest). Lines of YAML added: ~100. Lines of Markdown added: ~800 across four skills. The work shipped in 1–2 days. The harness team was not involved.

🍷 VII. A Day in the Life

Three workflows, one harness, end-to-end. The same dispatch loop runs all three.

Workflow A — software development

Operator submits an epic: "Add /healthz endpoint variant returning the same body as /health." The dev recipe (`development-kanban`) activates. Architect scopes the epic into one feature and one story. Story role flips: implementer claims, edits code, /verify; reviewer claims, peer-reviews, /verify; tester claims, runs `npm run build` and verifies the `visibleAt[]` entry (a `kind: api-page` assertion against `GET /healthz` returning 200), /verify; releaser claims, merges branch to develop, /complete. Outcome: the epic ships in 25 minutes wall-clock, with a forensic trail in `pipeline_outbox` and `story_log_entries`.

Workflow B — marketing campaign launch

Operator submits an epic: "Launch Q3 campaign for the Athena product." The marketing recipe (`campaign-launch`) activates. Strategist drafts the campaign brief. Story role flips: copywriter drafts copy variants, /verify; brand-reviewer evaluates against brand voice, /verify (or routeBack to copywriter on `tone-mismatch`); distributor schedules to Postiz via the `postiz-publish` ingredient, /complete. Outcome: campaign assets queued for publication, with the same outbox audit trail.

Workflow C — customer support escalation

Inbound ticket creates an epic. The support recipe (`escalation-flow`) activates. Triage-agent classifies severity, /verify. Technical-resolver attempts a fix, /verify. Customer-comms drafts a resolution message, /verify (sentiment-score ingredient evaluates negative sentiment → routeBack to qa-supervisor). qa-supervisor reviews and approves, /complete. Outcome: ticket resolved, customer messaged, audit trail complete.

✦ THE PUNCHLINE

The dispatcher loop is the same in all three. The story state machine is the same. The outbox writes the same event types. The dashboard renders the same diagnostic card. The reviewer's quality bar is different — but that's because the reviewer skill body is different. The harness is identical. *It does not know which workflow it is running.*

🔧 VIII. Scaling — Why the Harness Stays the Same Size

As the catalog of workflows grows, the harness does not grow. The math holds because the harness has no domain knowledge to grow into.

1×

One workflow. Harness ~6,000 LOC. One recipe, five skills, twenty pages of doctrine.

5×

Five workflows. Harness ~6,000 LOC (*unchanged*). Five recipes, twenty-five skills. Each workflow team owns its Package; the harness team owns the runtime.

25×

Twenty-five workflows. Harness ~6,200 LOC (small additions for new ports — e.g. an `IObjectStorage` port for binary artifacts in a new domain). Twenty-five recipes, ~125 skills. Workflow catalog grew 25×; harness grew 3%.

100×

One hundred workflows. Harness ~6,500 LOC. New ports added as new integration categories arrive (`ICalendarConnector` , `ITranslator`). Workflow catalog grew 100×; harness grew 8%.

Why the math holds

The harness's responsibilities are bounded by doctrine: dispatch, I/O, context, safety, plugin routing. None of these scale with workflow count. Dispatch logic is the same whether one recipe or one hundred is registered. Context resolution is the same per-stage. Safety enforcement (auth, claim-token, ContractGate) is per-call. Plugin routing is per-Package, but the routing logic is constant — it does not branch on Package identity.

The catalog scales with the team that authors workflows, not the team that maintains the harness. A new team adopting the platform writes Packages on top; they do not need to read the harness. The harness team is one or two people for the life of the platform.

⚙️ EMPIRICAL BASELINE (THIS REPO, MAY 2026)

```

src/pipeline/           # orchestration core           ~1800 LOC
src/services/           # dispatcher, scoper, reaper ~1500 LOC
src/adapters/           # integration adapters       ~3200 LOC
src/ports/              # interface contracts       ~600 LOC
src/domain/             # StoryAggregate + L1         ~700 LOC
src/routes/             # HTTP boundary           ~2200 LOC
src/recipes/ + skills/  # declarative artifacts     grows freely
src/packages/           # installable Packages       grows freely

```

Harness footprint: ~10K LOC. Skills + recipes + packages: unbounded. The two scale independently.

🔑 IX. Operating Procedure — Extending the Factory

Adding a new workflow

1. Identify the role sequence and quality bars unique to your domain.
2. Create `packages/<your-domain>-suite/package.yaml`.
3. Author one `skills/<role>/SKILL.md` per role (12-section template, full schema).
4. Author `recipes/<workflow-name>.yaml` declaring stages, ingredients, gates, routeBacks.
5. Run the test fixture `verifyPackageIntegrity()` — it loads your manifest, scans your skills, validates every reference resolves.
6. Restart the server. Submit a smoke epic against your recipe. Watch it ship through the dashboard.

Adding a new integration

1. Identify the external system (e.g. a new LLM, a new calendar API).
2. Define the port: `src/ports/<capability>.port.ts`. Interface + discriminated error union.
3. Implement at least two adapters: production + test fake.
4. Register in DI container with appropriate lifecycle.
5. Reference the port from any Extension or Ingredient that needs the capability.
6. Write an ADR documenting the port contract and substitution invariants.

Composing workflows (the Pass-2 frontier)

The `EPIC-WORKFLOW-COMPOSITION` backlog item (see `docs/backlog/`) introduces typed handoff schemas, declarative triggers, and pass-back protocols. Once shipped, recipes can chain: dev recipe completion fires e2e-validation recipe; e2e-validation pass-back creates a fix story in the dev epic. The chain composition itself is data — declared in Package manifests, validated at boot, executed by a generic trigger evaluator.

What NOT to do

- Do not add domain knowledge to the dispatcher. If you find yourself writing `if (story.role === 'X')` in a harness file, stop. Move the conditional to a skill body or a routeBack rule.
- Do not add a new closed enum to `src/ports/` for an installable concept. `GateKind`, `RoleKind`, `OutputKind` — these stay open. Manifests + registries.
- Do not call other adapters directly from a service. Resolve through DI. The service depends on shapes, not on identities.
- Do not bypass the StoryAggregate. Every state mutation goes through the L1 aggregate so the L2 outbox writes the event in the same transaction.
- Do not skip the BootContextAssembler. Every dispatch begins with `assemble()`; the agent's first token of stdout is preceded by a fully-loaded envelope.

X. Closing

A workflow factory is not a workflow tool. It is the substrate on which workflow tools are authored. The substrate's value is its consistency: every workflow runs the same way through the same gates with the same audit trail and the same safety properties. The substrate's commitment is its boundary: it does not learn the workflows it runs; it does not need to.

This commitment buys three properties at scale. **The harness team stays small.** The platform's maintenance cost is fixed; only the catalog grows. **Workflow teams move independently.** A marketing team can ship a campaign workflow without coordinating with a software team shipping a dev workflow. Their Packages are isolated; the harness merges them at runtime. **The substrate compounds.** Every Extension installed by one workflow benefits every other workflow that opts in. `slack-notifications` shipped by the dev team becomes available to the marketing team the next day.

This is what an extensible workflow platform looks like when its doctrine holds. Five primitives, plural ports, plural adapters, one harness that stays small. Skills are code. Recipes are data. Packages are plugins. Extensions are cross-cutting capabilities. The harness orchestrates and validates — and beyond that, it does not know what kind of work it is doing. *Which is exactly why it can do anything.*

One harness. Many recipes. Every recipe a YAML file. Every capability a Package. Every contract a schema. Add a workflow in an afternoon. The factory floor never expands.



The messenger god knew nothing of the messages he carried. Which is exactly why he could carry any of them, anywhere, at any time.